



Seminario de Programación



C++ - Orientación a Objetos

Patricio Olivares - Wladimir Ormazábal

Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

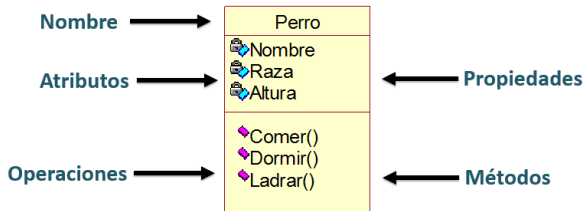
Objetos y Clases

Objetos y Clases

- **Concepto:** objetos en la realidad tienen *atributos* y *acciones*
- **Encapsulación:** definir una estructura de *datos* junto a sus *funciones* asociadas
- **Clase:** define un *tipo de dato abstracto* que contiene el *estado* (variables) y *métodos* (funciones)
- **Objeto:** una instancia de una clase (una variable del tipo definido en la clase)

Objetos y Clases

- **Concepto:** objetos en la realidad tienen *atributos* y *acciones*
- **Encapsulación:** definir una estructura de *datos* junto a sus *funciones* asociadas
- **Clase:** define un *tipo de dato abstracto* que contiene el *estado* (variables) y *métodos* (funciones)
- **Objeto:** una instancia de una clase (una variable del tipo definido en la clase)



Hands on: Videojuego (10 mins) [Parte 1]

- Piense en su videojuego favorito (con personajes)
- Considere ahora un personaje de ese videojuego
 1. Identifique **tres** atributos de ese personaje (variables)
 2. Identifique **tres** métodos de ese personaje (funciones)
 3. ¿De qué tipo son los atributos que usted eligió?
 4. ¿Alguno de sus métodos tienen parámetros? (cuales)
 5. ¿Su jugador pertenece a algún *tipo* (clase)?
- **Seleccionaré a dos estudiantes para que los explique brevemente**



Ejemplo: Among Us!

Ejemplo: Among Us!

```
In [ ]: // Tripulante  
  
char nom[100];  
float posicion;  
int avatar;  
  
void reportar_cuerpo(int quien);  
void moverse(int direccion);  
void hacer_tarea(float posicion);
```


Ejemplo: Among Us!

```
In [ ]: // Tripulante  
  
char nom[100];  
float posicion;  
int avatar;  
  
void reportar_cuerpo(int quien);  
void moverse(int direccion);  
void hacer_tarea(float posicion);
```



Ejemplo: Among Us!

```
In [ ]: // Tripulante  
  
char nom[100];  
float posicion;  
int avatar;  
  
void reportar_cuerpo(int quien);  
void moverse(int direccion);  
void hacer_tarea(float posicion);
```



- ¿Qué tenemos que hacer si queremos crear otro tripulante?
- ¿Qué tenemos que hacer si queremos crear un impostor?
- **Las clases encapsulan estas definiciones para facilitar su uso!**

Clases en C++

Clases en C++

- Los **atributos** se crean como variables internas en la clase
- Los **métodos** se crean como funciones internas en la clase
- La **declaración** de clase, al menos requiere el **prototipo** de los métodos. La implementación puede ir en otro archivo.
- Adicionalmente, C++ establece dos métodos fundamentales: constructor y destructor.
 - **Constructor:** Método de inicialización, se ejecuta inmediatamente después de crear un objeto (pueden ser varios)
 - **Destructor:** Método de destrucción, se ejecuta antes de liberar memoria (solo uno!)
- Existe un **control de acceso** (public/private) que explicaremos más adelante.
- **Sintáxis:** definición similar a struct (pero admite funciones)


```
In [ ]: class Animal {
public:
    //Prototipos de constructores:
    Animal(): edad_hijos(NULL), n_hijos(0) {}
    Animal(std::string i_nombre);

    ~Animal(); //Prototipo de destructor

    //Prototipos de metodos:
    void asigna_edad_hijo(int i, int edad);
    int edad_hijo(int i);

    //Implementacion de metodo:
    void crea_hijos(int num) {
        n_hijos = num <= 0 ? 0 : num;
        if(edad_hijos) delete[] edad_hijos;
        edad_hijos = num ? new int[num] : NULL;
    }

    //Atributos:
    std::string nombre;

private:
    int *edad_hijos;
    int n_hijos
};
```

Cabeceras e Implementación

- Algunos métodos solo están como **prototipos** (implementación postergada)
- Se pueden implementar **fuera de la definición** de clase.
- Se debe agregar la **referencia a la clase** (constructores también).
- La clase es un tipo, por lo que su **definición** va los .h o .hpp (headers)
- Las **implementaciones** van en un .cpp

```
In [ ]: #include <iostream>
#include "animal.h"

Animal::Animal(std::string i_nombre) : n_hijos(0), edad_hijos(NULL), nombre(i_nombre) {}

Animal::~Animal() {
    if(edad_hijos)
        delete[] edad_hijos;
}

int Animal::edad_hijo(int i) {
    if(i < 0 || i >= this->n_hijos)
        return 0;
    return edad_hijos[i];
}

void Animal::asigna_edad_hijo(int i, int edad) {
    if(i < 0 || i >= this->n_hijos) {
        std::cerr << "Indice " << i << " fuera de limites..." << std::endl;
        return;
    }
    edad_hijos[i] = edad;
}
```

```

In [ ]: #include <iostream>
#include "animal.h"

Animal::Animal(std::string i_nombre) : n_hijos(0), edad_hijos(NULL), nombre(i_nombre) {}

Animal::~~Animal() {
    if(edad_hijos)
        delete[] edad_hijos;
}

int Animal::edad_hijo(int i) {
    if(i < 0 || i >= this->n_hijos)
        return 0;
    return edad_hijos[i];
}

void Animal::asigna_edad_hijo(int i, int edad) {
    if(i < 0 || i >= this->n_hijos) {
        std::cerr << "Indice " << i << " fuera de limites..." << std::endl;
        return;
    }
    edad_hijos[i] = edad;
}

```

Notar método de inicialización en constructor: listas de inicialización, asigna en la creación del atributo (sin importar orden, se inicializan valores según orden de declaración en la clase).

Hands on: Videojuego (20 mins) [Parte 2]

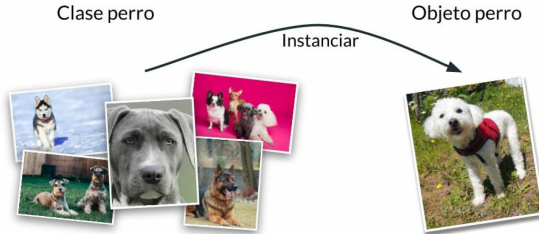
- Defina una clase en C++ para el personaje que usted eligió sin implementar sus métodos (en un archivo .h)
- No se preocupe aún por los parámetros, si es algo simple inclúyalos, si no omítalos.
- Haga una implementación simple que imprima por pantalla lo que debería hacer esa función (en un .cpp)
- Compile su .cpp exitosamente
- **Seleccionaré a dos estudiantes para que explique brevemente**



Creación de Objetos en C++

- Para crear objeto: **new**; para borrar **delete**
- Para crear arreglo: **new[]**; para borrar **delete[]**
- Acceso a campos con operador **.** (instancia) y **->** (puntero)

Un objeto es la **instancia de una clase**.



Creación de Objetos en C++

Creación de Objetos en C++

```
In [ ]: #include "animal.h"
#include <iostream>

int main() {
    Animal un_animal; //Un objeto (estático/stack).
    Animal *referencia_a_otro = new Animal; //Un puntero al objeto.
    Animal *referencia_a_otro_mas = new Animal("Marcos");

    //Un puntero a un arreglo dinámico de 10 animales:
    Animal *un_arreglo = new Animal[10];

    //Acceso a metodo de objeto:
    un_animal.nombre = "Marcos Z.";

    //Acceso a atributo de objeto (algun error?):
    std::cout << referencia_a_otro->n_hijos << std::endl;
    std::cout << referencia_a_otro_mas->nombre << std::endl;

    //Borrado de puntero (se llama a destructor)
    delete referencia_a_otro;
    delete[] un_arreglo;
```

```
Animal un_error(); //Analizador sintáctico interpreta  
//como prototipo de función.  
return 0;
```

```
}
```

Modificadores de acceso

- Determinan la accesibilidad a los elementos de la clase.
- **public**: Cualquiera puede acceder.
- **protected**: Sólo hijos (herencia) y miembros de la clase pueden acceder.
- **private**: Sólo miembros de la clase pueden acceder.

Acceso	Clase	Subclase	TODOS
public	✓	✓	✓
protected	✓	✓	✗
private	✓	✗	✗

Modificadores de acceso

- Determinan la accesibilidad a los elementos de la clase.
- **public**: Cualquiera puede acceder.
- **protected**: Sólo hijos (herencia) y miembros de la clase pueden acceder.
- **private**: Sólo miembros de la clase pueden acceder.

Acceso	Clase	Subclase	TODOS
public	✓	✓	✓
protected	✓	✓	✗
private	✓	✗	✗

```
In [ ]: class Animal {  
    public:  
        Animal();  
        ~Animal();  
        std::string nombre; //Todos pueden acceder  
    protected:  
        int *edad_hijos; //Solo miembros de la clase y derivados  
    private:  
        int n_hijos; //Solo miembros de la clase  
};
```

Hands on: Videojuego (15 mins) [Parte 3]

- Cree un main con un arreglo de 4 personajes (consola con 4 controles)
- Proteja sus variables de estado (atributos) y cree un método para asignar una de ellas
- Itere por los 4 personajes para ingresar estos valores
- **Seleccionaré a dos estudiantes para que explique brevemente**



Among Us: Tripulantes e Impostores

- En este juego existen **2 casos** de jugadores
- Algunas **acciones son las mismas** entre ellos (e.g. moverse)
- Otras son distintas (e.g. asesinar, hacer tareas)
- **Escribir dos veces el mismo código es mala idea!**
- **Para reutilizar código, tenemos que definir una jerarquía de clases**

Among Us: Tripulantes e Impostores

- En este juego existen **2 casos** de jugadores
- Algunas **acciones son las mismas** entre ellos (e.g. moverse)
- Otras son distintas (e.g. asesinar, hacer tareas)
- **Escribir dos veces el mismo código es mala idea!**
- **Para reutilizar código, tenemos que definir una jerarquía de clases**



Herencia

- Una clase puede **extender** lo que hace otra clase
- **Hereda** su estado y su comportamiento (atributos y métodos)
- Relación **jerárquica** o de parentesco (padres e hijos)

Herencia

- Una clase puede **extender** lo que hace otra clase
- **Hereda** su estado y su comportamiento (atributos y métodos)
- Relación **jerárquica** o de parentesco (padres e hijos)

```
In [ ]: class Cocodrilo : public Animal {
public:
    Cocodrilo();
    int numero_dientes;
};

class Pez : public Animal {
public:
    Pez();
    int numero_de_escamas;
};

class PezGlobo : public Pez {
public:
    PezGlobo();
    double diametro_inflado;
};
```

Herencia: Ventajas

- **Reutilización de código:**
 - Utilización de una clase base para nuevas clases
 - No hace falta comprender el código de fuente de la clase base, sólo saber lo que hace (encapsulamiento)
- **Extensibilidad:**
 - Programas fácilmente ampliables
 - De una clase base se pueden derivar varias clases que tengan una interfaz común, (aunque las acciones sean diferentes).

Herencia: Sintaxis

Herencia: Sintaxis

```
In [ ]: class #nombre_clase_especifica: [#acceso] #nombre_clase_heredada {  
        // Cuerpo de la declaraci on de la clase  
};
```

- Los elementos **private** son inaccesibles para la clase derivada, sea cual sea el acceso.
- Modificadores de acceso
 - **public**: todos los elementos public y protected de la clase base quedan igual en la derivada
 - **private**: todos los elementos public y protected de la clase base serán elementos private de la clase derivada
 - **protected**: todos los elementos public y protected de la clase base serán elementos protected de la clase derivada.

Elementos de Clase

- Un elemento (atributo o método) con el modificador **static** pasa a ser un elemento **general de la clase**, no de la instancia.
- Significa que **no requieren una instancia** para tener acceso a ellos.
- C++ lo toma como **definición**. Se declara fuera de la clase.
- Para los métodos, también significa que **no pueden manipular elementos de una instancia**.
- Los **operadores de acceso** son los mismos que para **namespace**, respectivamente, usando el **nombre de la clase**.
- **No abuse de este modificador!**

Elementos de Clase

- Un elemento (atributo o método) con el modificador **static** pasa a ser un elemento **general de la clase**, no de la instancia.
- Significa que **no requieren una instancia** para tener acceso a ellos.
- C++ lo toma como **definición**. Se declara fuera de la clase.
- Para los métodos, también significa que **no pueden manipular elementos de una instancia**.
- Los **operadores de acceso** son los mismos que para **namespace**, respectivamente, usando el **nombre de la clase**.
- **No abuse de este modificador!**

In []:

```
class C {  
public:  
    static int a,b; //Definicion incompleta  
    static void f();  
}  
  
//Declaracion;  
int C::a;  
int C::b = 2;  
//Acceso:
```

```
C::a = C::b + 5;
```

```
void C::f()  
{  
    a = 1;  
}
```

Sobrecarga de Funciones

- Cuando una clase derivada y la clase heredada tienen algún **elemento con el mismo nombre**, el compilador debe **elegir** cual ocupa
- El compilador de C++ elige siempre el elemento de la **clase derivada** (sobrecarga u *overriding*).

Sobrecarga de Funciones

- Cuando una clase derivada y la clase heredada tienen algún **elemento con el mismo nombre**, el compilador debe **elegir** cual ocupa
- El compilador de C++ elige siempre el elemento de la **clase derivada** (sobrecarga u *overriding*).

```
In [ ]: class Animal {
        public:
            Animal();
            Animal(std::string i_nombre);
            ~Animal();
            void come();
            std::string nombre;
        protected:
            int *edad_hijos;
            int n_hijos;
    };

    class Cocodrilo : public Animal {
    public:
        Cocodrilo(std::string i_nombre);
        int numero_dientes;
        void come();
    };

```

```
};  
  
int main() {  
    Animal a("Marcos");  
    Cocodrilo Juancho("Juancho");  
    .....  
    a.come(); //Metodo de Animal.  
    Juancho.come(); // de Cocodrilo.  
}
```

Sobrecarga de Funciones

- Cuando una clase derivada y la clase heredada tienen algún **elemento con el mismo nombre**, el compilador debe **elegir** cual ocupa
- El compilador de C++ elige siempre el elemento de la **clase derivada** (sobrecarga u *overriding*).

```
In [ ]: class Animal {
        public:
            Animal();
            Animal(std::string i_nombre);
            ~Animal();
            void come();
            std::string nombre;
        protected:
            int *edad_hijos;
            int n_hijos;
    };

    class Cocodrilo : public Animal {
    public:
        Cocodrilo(std::string i_nombre);
        int numero_dientes;
        void come();
    };
```



```
};  
  
int main() {  
    Animal a("Marcos");  
    Cocodrilo Juancho("Juancho");  
    .....  
    a.come(); //Metodo de Animal.  
    Juancho.come(); // de Cocodrilo.  
}
```

```
In [ ]: #Salida:  
        #Marcos come como un animal!!  
        #Juancho come como un cocodrilo!!
```

Métodos Virutales

- **Coerción en OO:** puntero de tipo clase padre contiene referencia a una instancia de la clase hijo.
- **Problema:** ¿Cómo accedo a un método suplantado de la clase hijo?
- **Métodos virtuales:** Forma de acceder a los métodos de las subclases desde un puntero a la clase padre.
- Si un método es **virtual** en la clase padre, se **llamará al método de la clase derivada** (si existe).
- Si no es virtual, se llamará al método de **la clase padre**.

```
In [ ]: class Animal {
        public:
            virtual void come() {
                cout << "Yo como como un animal!\n";
            }
    };

    class Lobo : public Animal {
    public:
        void come() {
            cout << " Yo como como un lobo!\n";
        }
    };

    class Pez : public Animal {
    public:
        void come() {
            cout << " Yo como como un pez!\n";
        }
    };

    class OtroAnimal : public Animal{

    };

    int main() {
        Animal *unAnimal[4];
        unAnimal[0] = new Animal();
        unAnimal[1] = new Lobo();
        unAnimal[2] = new Pez();
```

```
unAnimal[3] = new OtroAnimal();  
for(int i = 0; i < 4; i++)  
    unAnimal[i]->come();  
for (int i = 0; i < 4; i++)  
    delete unAnimal[i];  
return 0;  
}
```

```
## Salida con virtual:  
#Yo como como un animal!  
#Yo como como un lobo!  
#Yo como como un pez!  
#Yo como como un animal!  
##Salida sin virtual:  
#Yo como como un animal!  
#Yo como como un animal!  
#Yo como como un animal!  
#Yo como como un animal!
```

